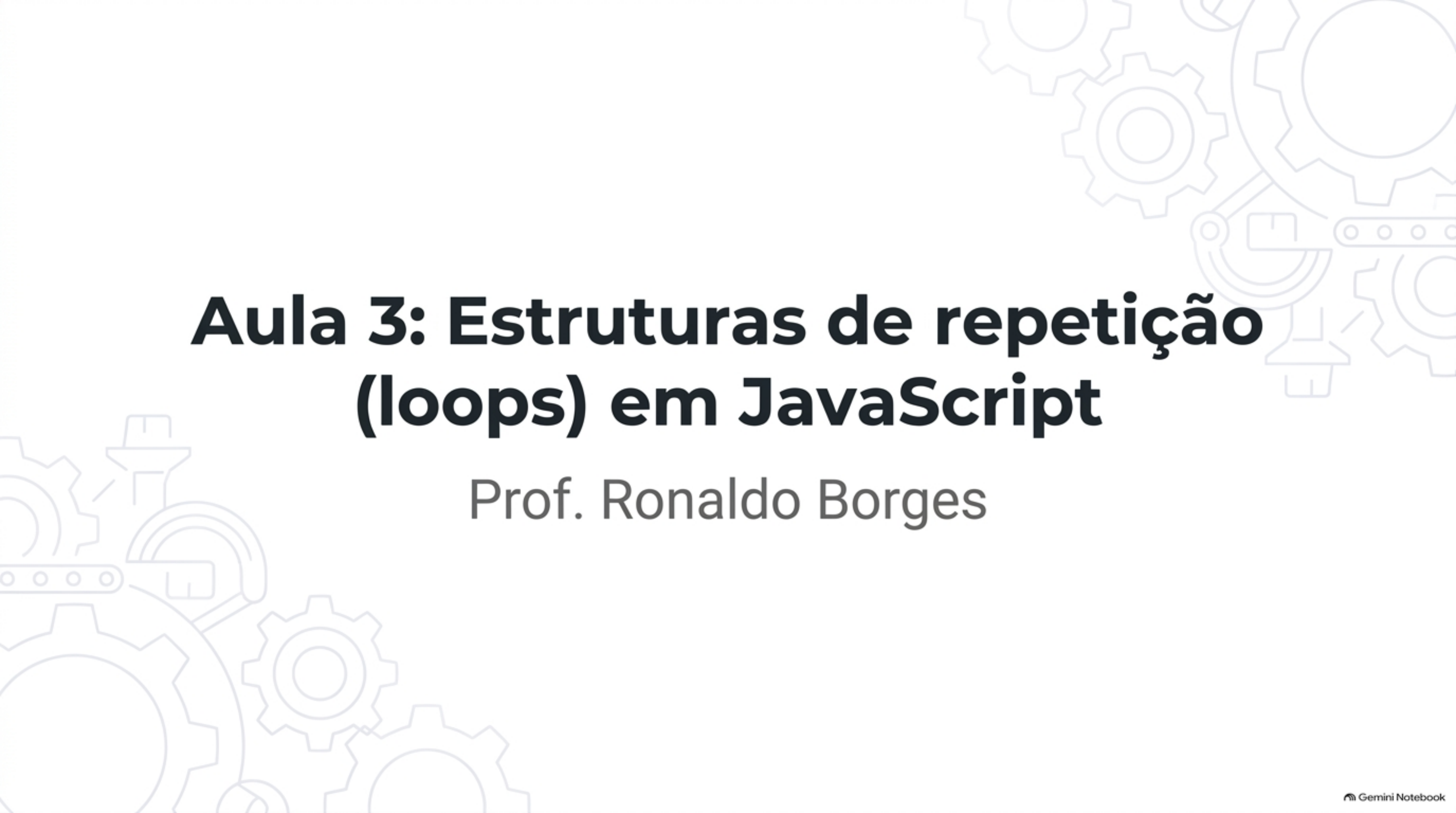




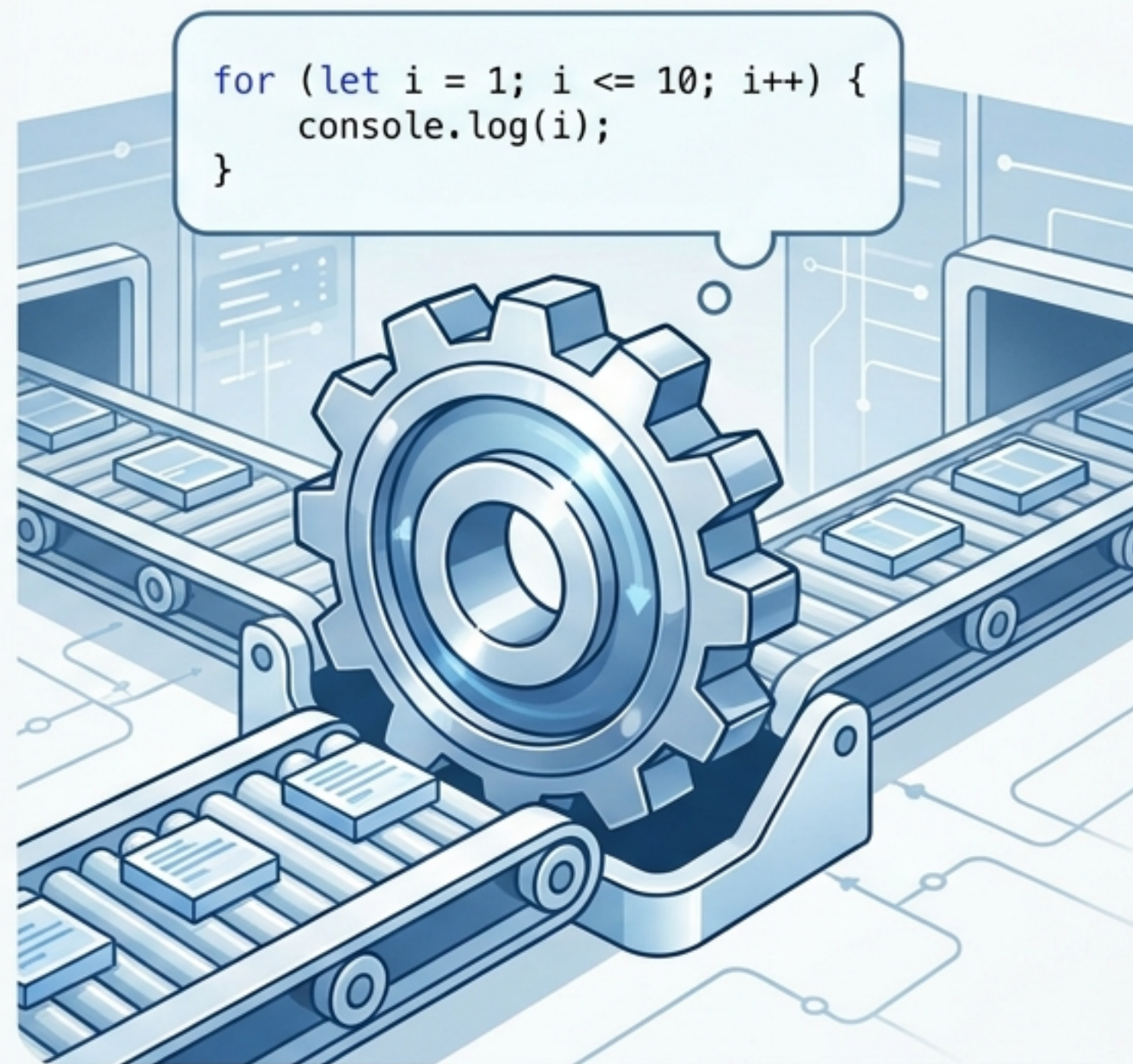
Aula 3: Estruturas de repetição (loops) em JavaScript

Prof. Ronaldo Borges

A Necessidade da Automação (O Problema)

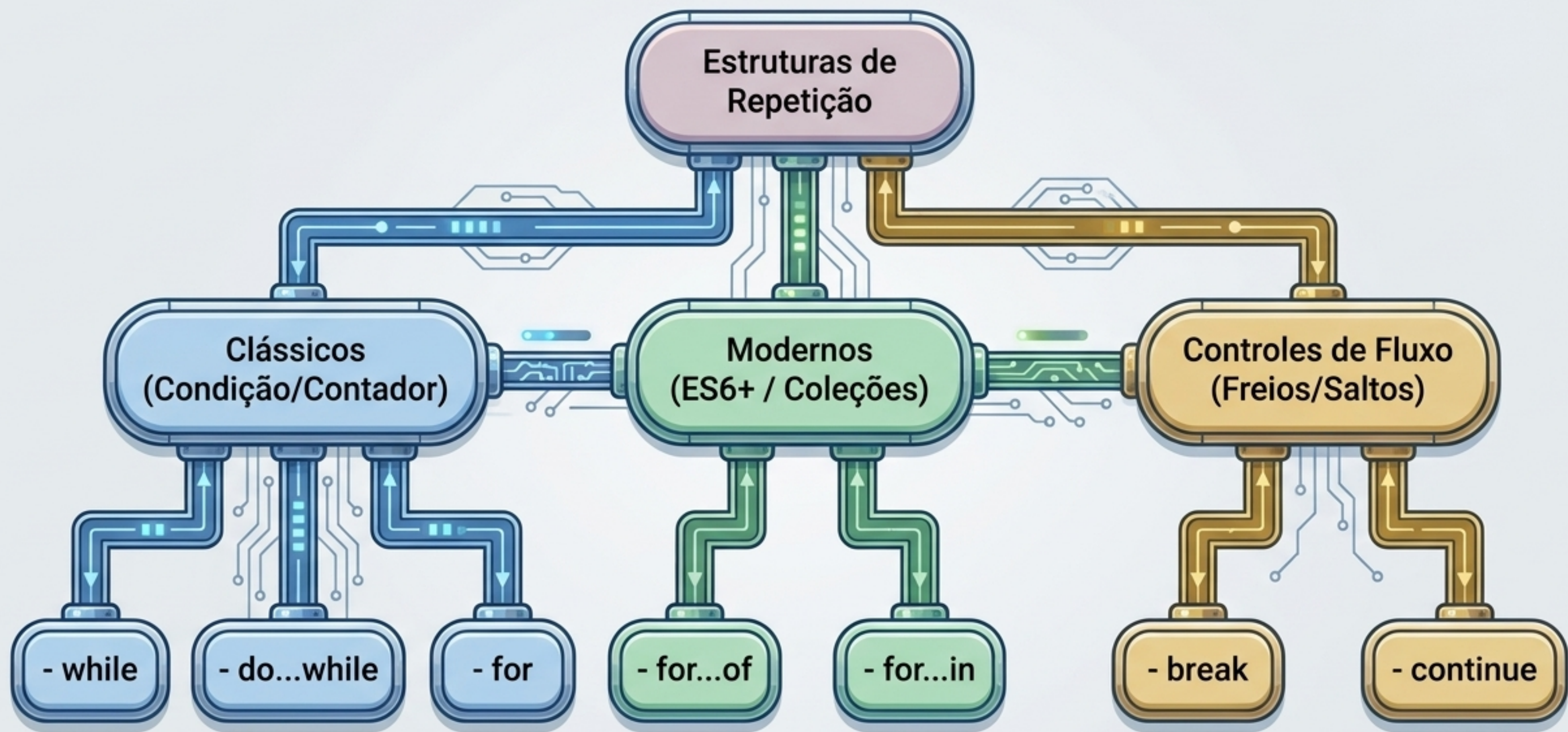


Trabalho braçal, repetitivo e propenso a erros.

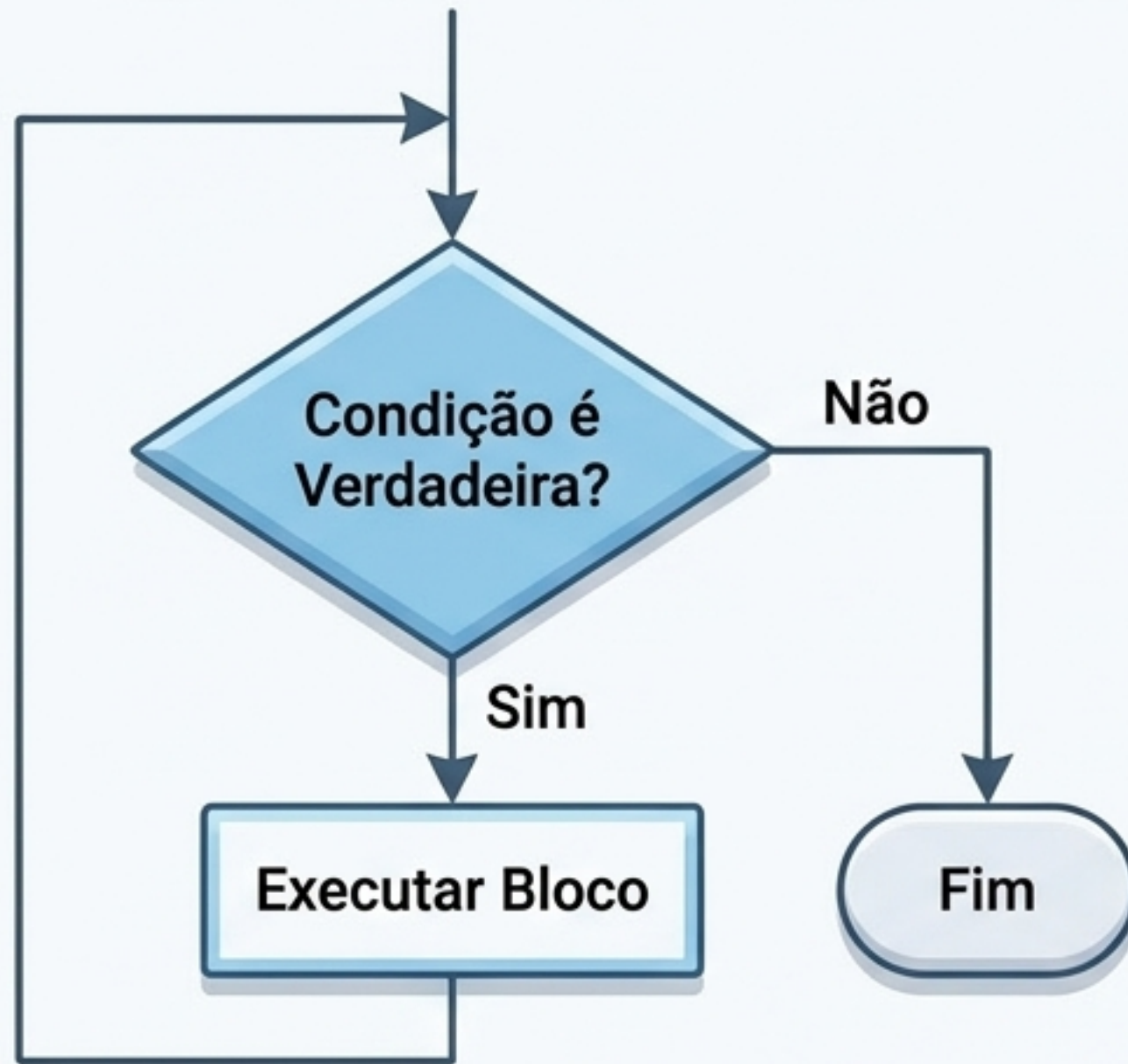


Automação: O código trabalha para você.

O Ecossistema de Laços no JavaScript



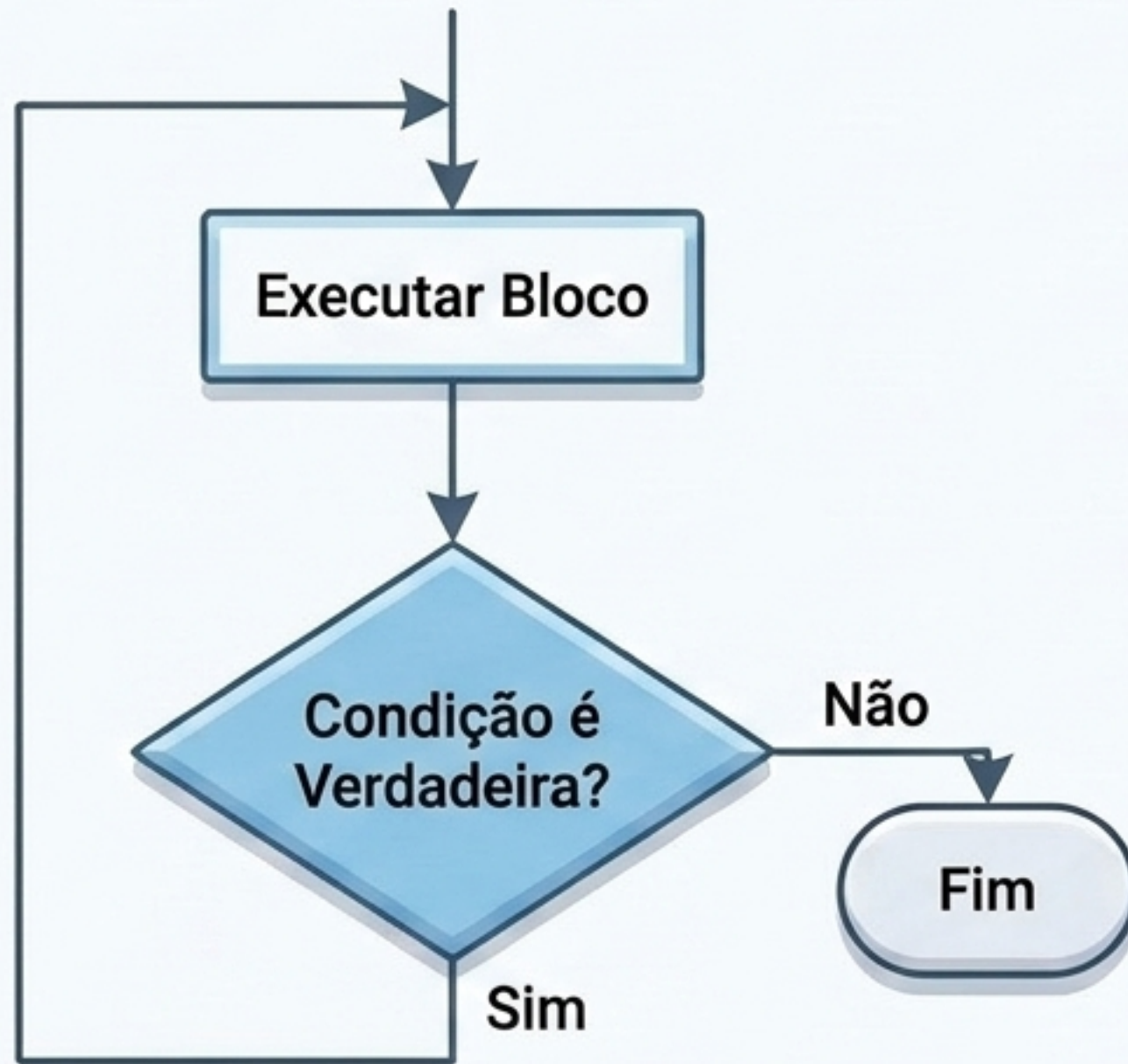
Laço while (Condicional Prévia)



```
let i = 0;  
while (nomePesquisado == null && i  
    < nomes.length) {  
    if (nomes[i] == 'Melequento') {  
        nomePesquisado = nomes[i];  
    }  
    i++;  
}
```

A verificação ocorre no início. Se a condição for falsa logo de cara, o bloco interno nunca será executado.

Laço do...while (Execução Garantida)



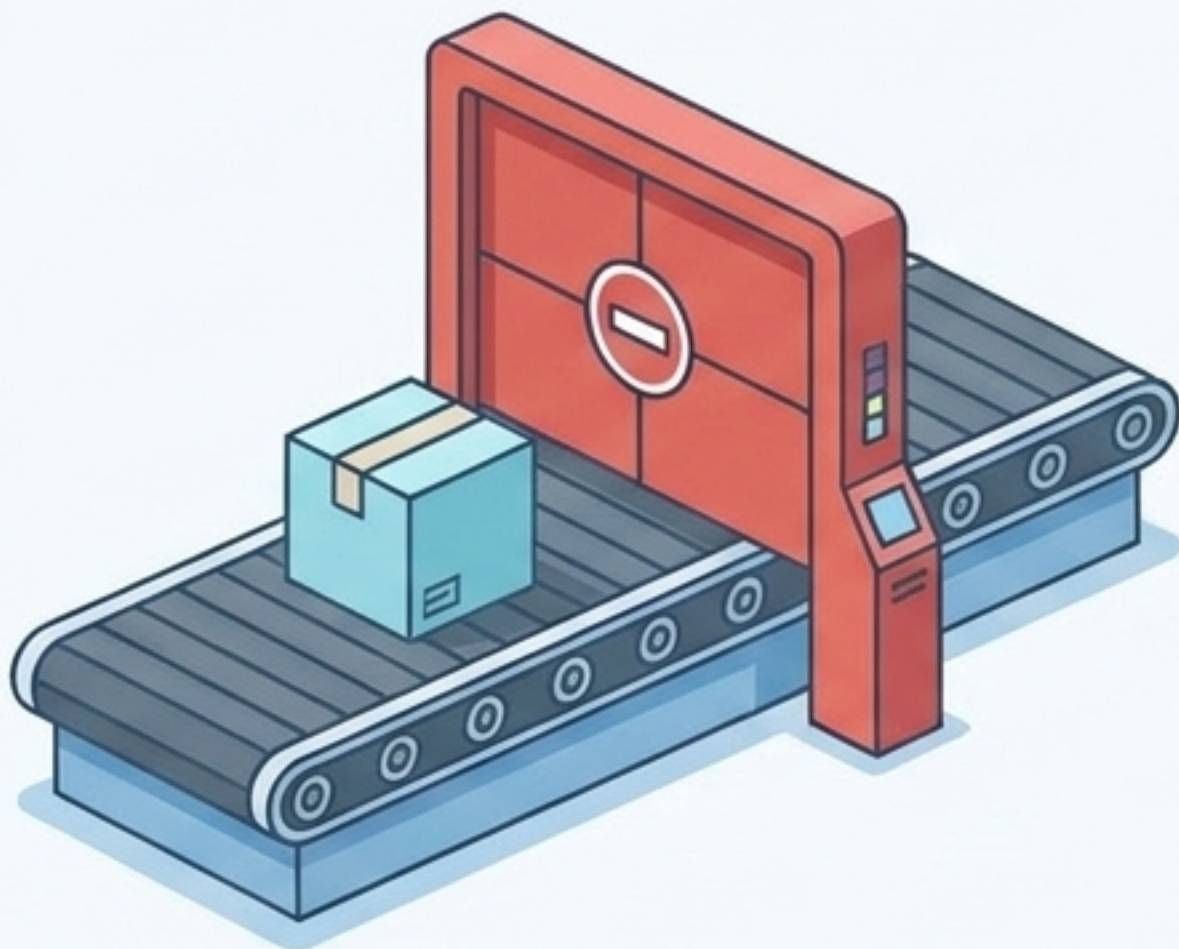
```
let i = 1;
do {
  if (i % 2 !== 0) {
    console.log(i);
    console.log(i);
  }
  i++;
} while (i <= 20);
```

A avaliação é feita no final. O código interno é executado obrigatoriamente pelo menos uma vez.

Matriz Visual: while vs do...while

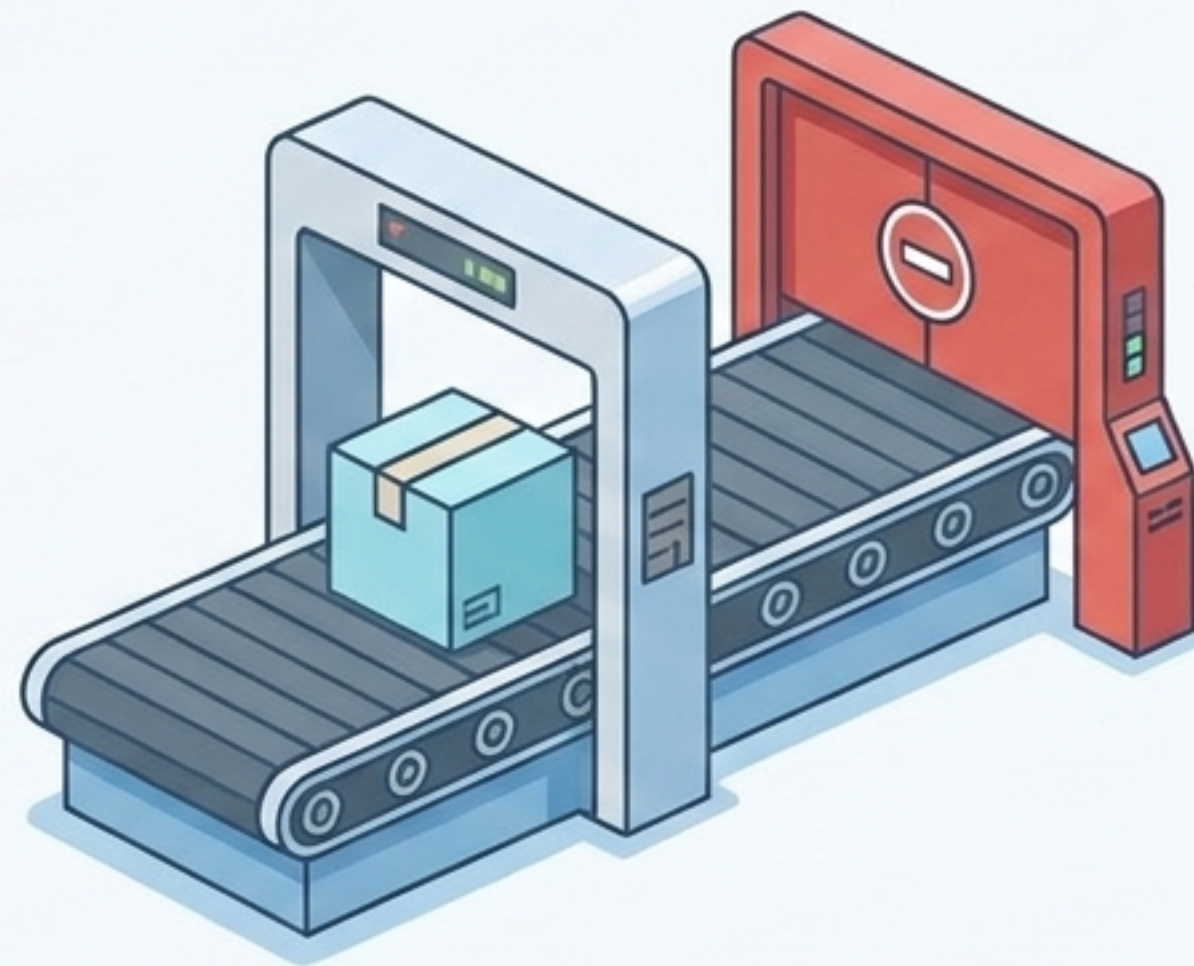
E se a condição já nascer FALSA?

while



0 Execuções. O bloco é totalmente ignorado.

do...while



1 Execução. O bloco roda antes da porta se fechar.

A Anatomia do Laço for Clássico

1. Inicialização: Onde começamos.
Cria a variável de controle antes
do laço iniciar.

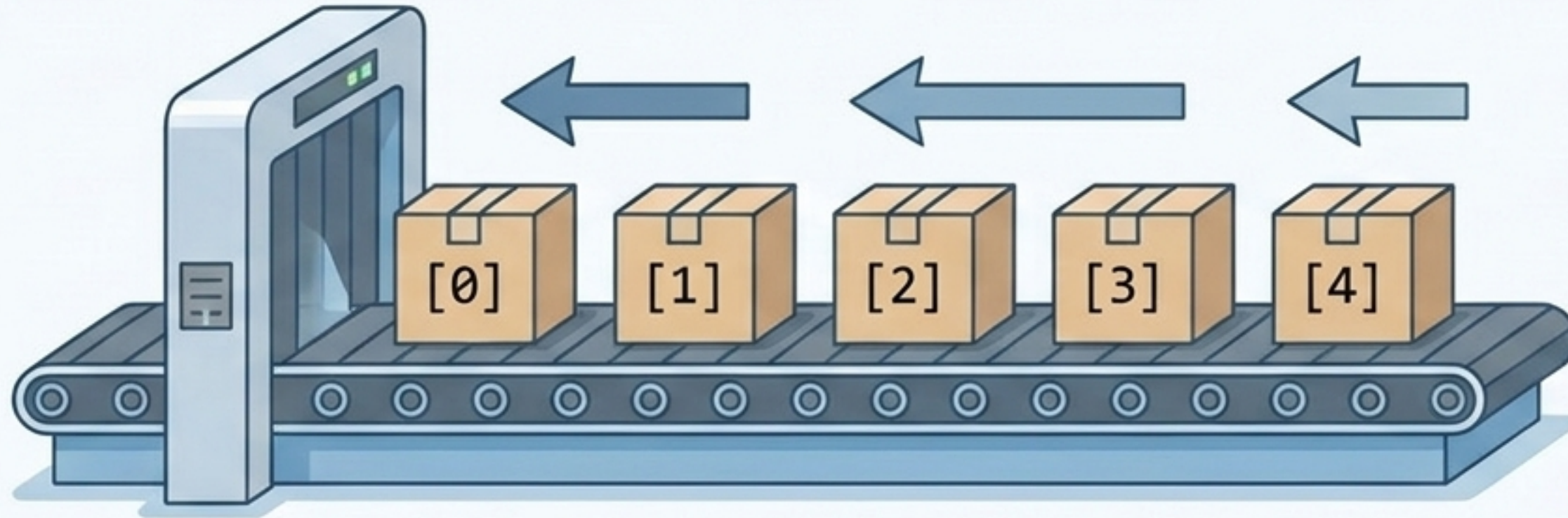
2. Condição: A regra de parada.
Avaliada rigorosamente antes de
cada ciclo.

```
for (let i = 0; i < 10; i++) {  
    ...  
}
```

3. Incremento: O passo.
Executado após o fim de cada ciclo,
preparando para o próximo.

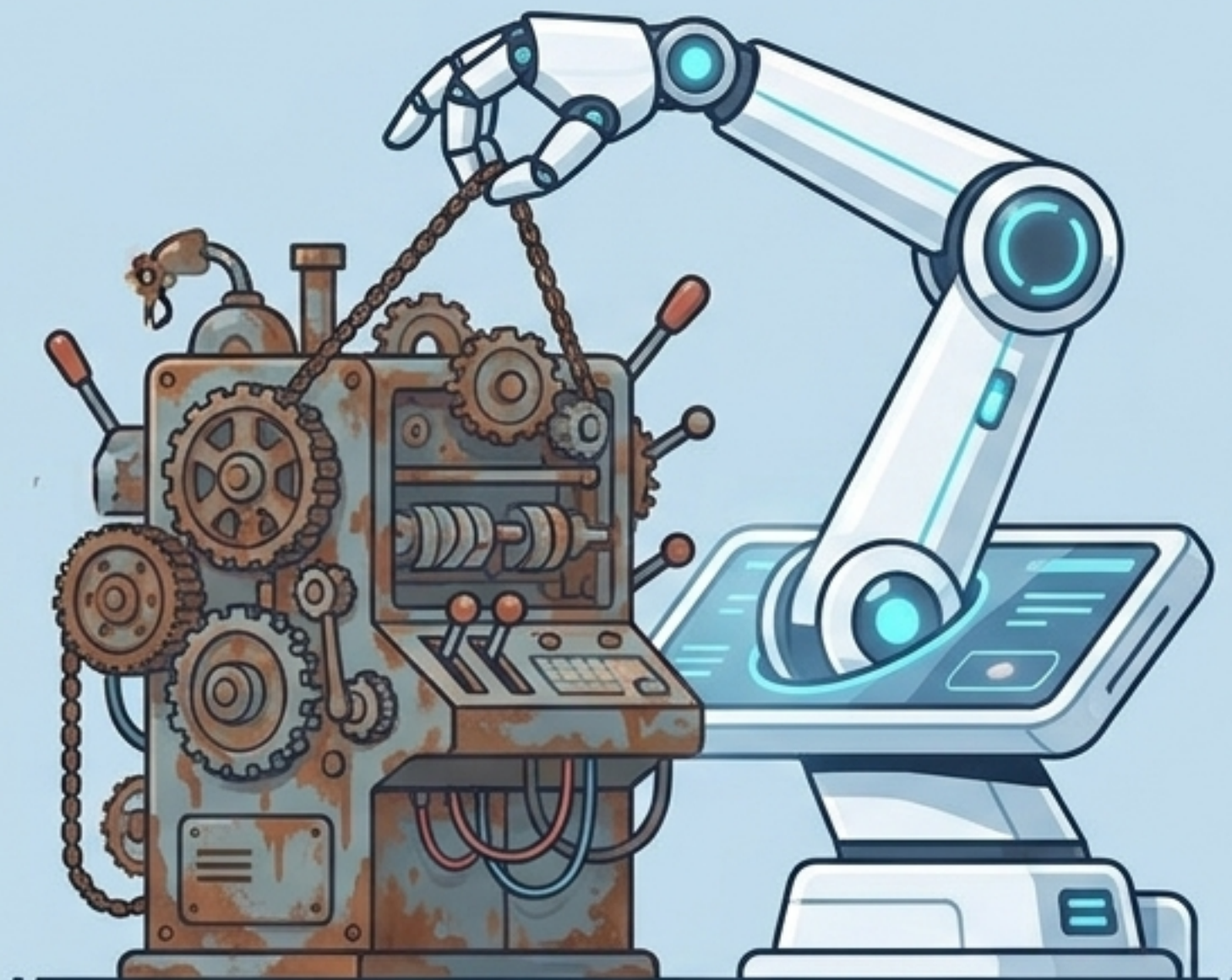
É a ferramenta ideal quando você sabe exatamente a
quantidade de repetições necessárias de antemão.

O Laço for na Prática (Percorrendo de Trás para Frente)



```
var nomes = ['João', 'Maria', 'Aurora', 'Branca', 'Melevola'];  
  
// Iniciamos no último índice e decrementamos (i--)  
for (let i = nomes.length - 1; i >= 0; i--) {  
  console.log(nomes[i]);  
}
```

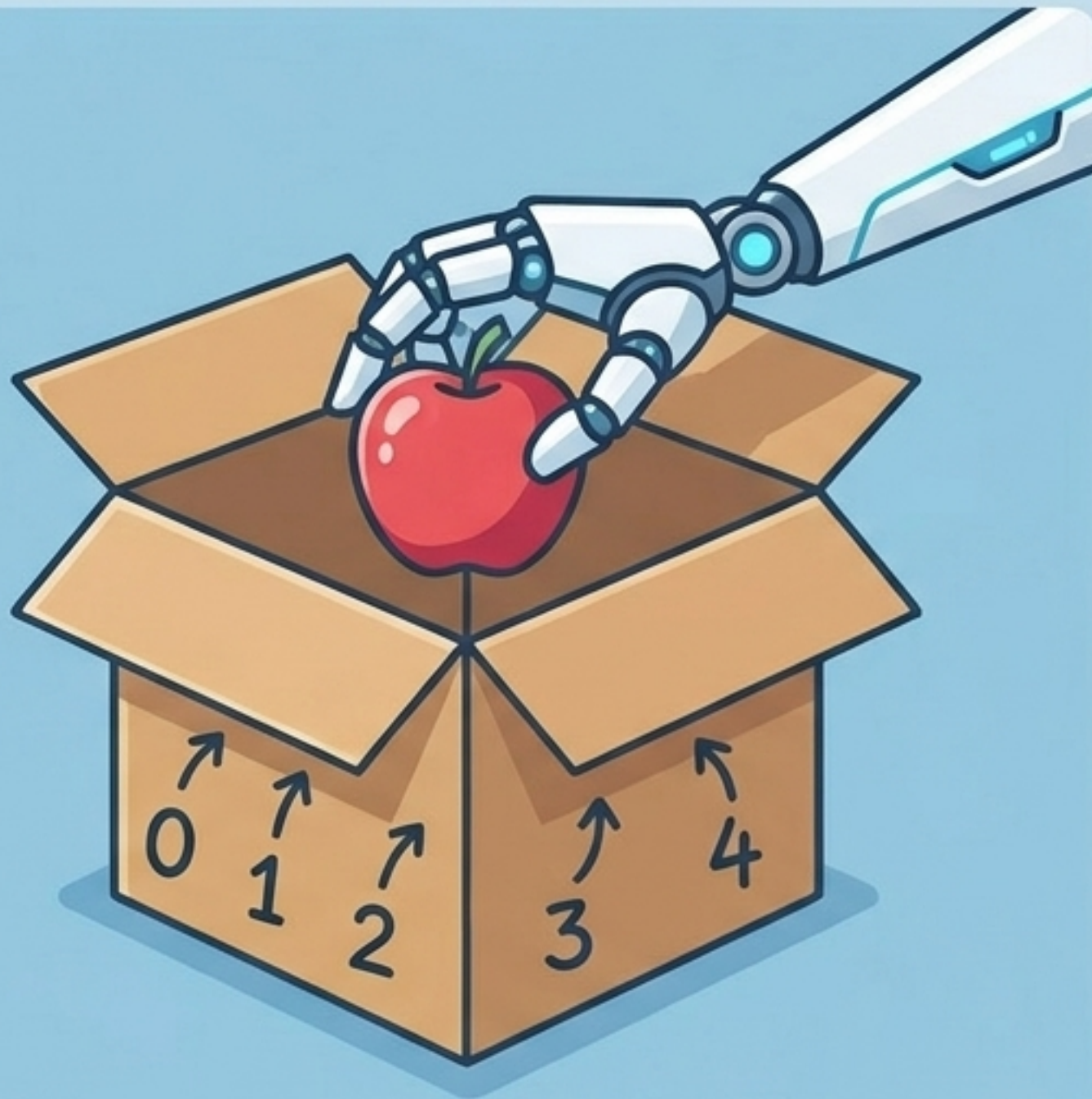
O controle preciso dos três parâmetros permite manipular o fluxo da esteira livremente.



A Evolução ES6+ (Laços Modernos)

O JavaScript evoluiu. Percorrer Arrays e Objetos não exige mais controle manual de índices. O ECMAScript 2015 introduziu formas mais simples e declarativas de iterar sobre coleções de dados.

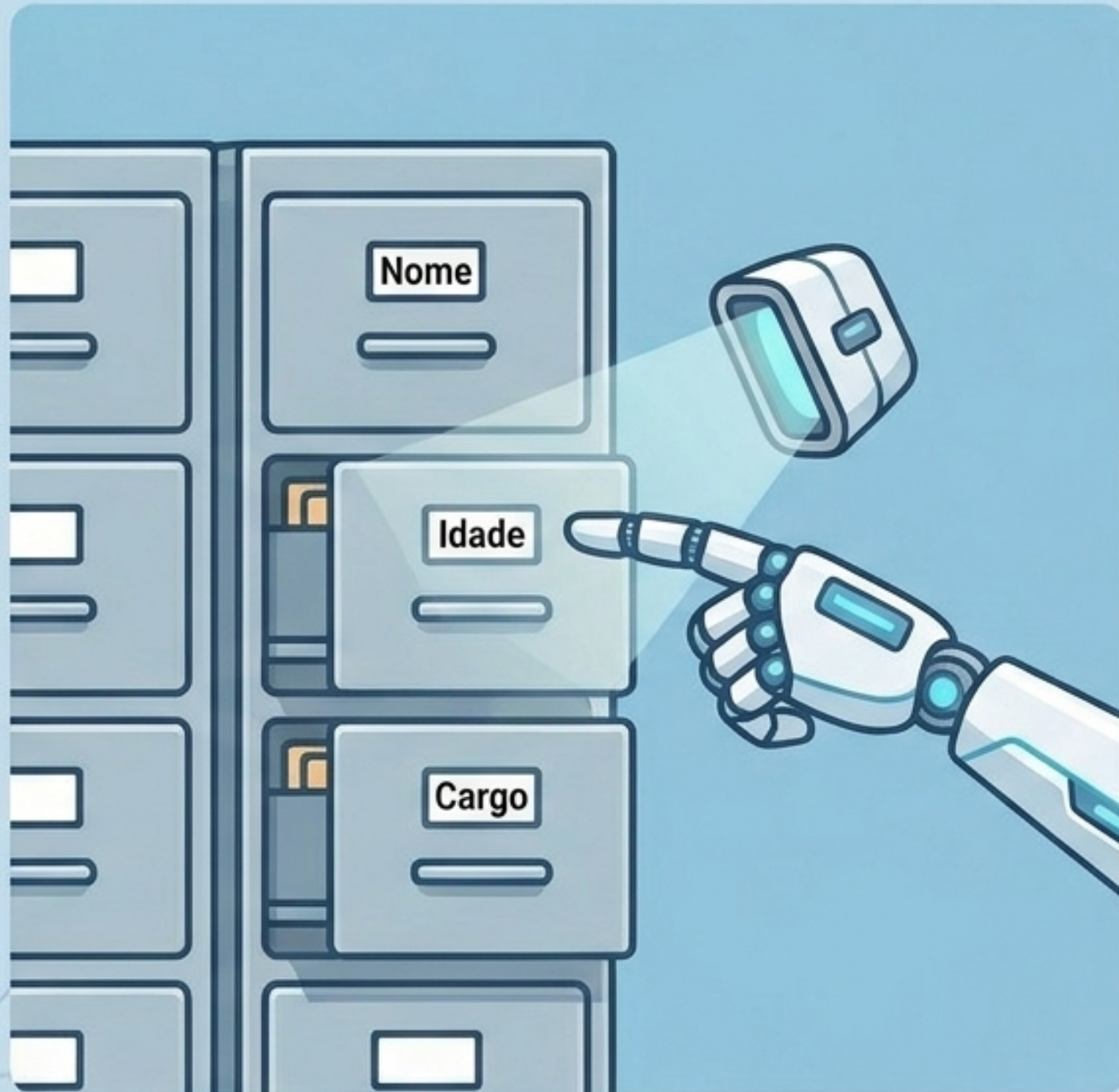
Laço for...of (Iterando Valores)



Criado para iterar diretamente sobre os valores de coleções de dados (Arrays, Strings, Maps).

```
let employeeArray = [1001, 'Sam', true, false];  
  
for (let item of employeeArray) {  
    console.log(item); // Imprime o valor direto  
}
```

Laço for...in (Iterando Chaves/Propriedades)



Utilizado para percorrer as propriedades enumeráveis (as chaves ou índices) de um Objeto.

```
let employee = { firstName: 'Sam',  
  lastName: 'Joy', pinCode: 411010 };  
  
for (let prop in employee) {  
  console.log(prop + ':' + employee[prop]);  
}
```

Matriz de Diagnóstico: Qual laço usar?

Laço	Foco de Iteração	Melhor Caso de Uso	Cuidado
for	Controle Exato	Contagens matemáticas e matrizes complexas	Requer gestão manual de índices
while	Condições Lógicas	Repetições imprevisíveis ou busca até encontrar	Risco de loop infinito
for...of	Valores	Arrays, Strings, Mapas (Coleções iteráveis)	Não funciona em Objetos literais
for...in	Chaves (Keys)	Inspecionar propriedades de um Objeto	Itera sobre propriedades herdadas (prototype)

Controle de Fluxo: A instrução break (O Freio)



```
var animais = ['peixe', 'baleia', 'tubarão', 'lula'];  
  
for (var i = 0; i < animais.length; i++) {  
  if (animais[i] == 'tubarão') {  
    console.log('Encontrado!');  
    break; // Interrompe e encerra imediatamente  
           o laço  
  }  
}
```

Direciona o fluxo do programa para a primeira linha de código subsequente ao loop, economizando processamento.

Controle de Fluxo: A instrução continue (O Salto)



```
var flores = ['rosa', 'amarelo', 'vermelho'];  
var numeroDeFlores = 0;  
  
for (var i = 0; i < flores.length; i++) {  
  if (flores[i] == 'amarelo') {  
    continue; // Salta a iteração atual (ignora  
              as amarelas)  
  }  
  numeroDeFlores++;  
}
```

Interrompe apenas a passagem (iteração) atual, saltando as linhas restantes do bloco e avançando direto para o próximo ciclo.

Referências Bibliográficas

BIFFI, Kelvin Baumhardt. JavaScript Básico ao Avançado. [S.l.: s.n.], [20--].

GRONER, Loiane. Estruturas de dados e algoritmos com JavaScript. São Paulo: Novatec, 2018.

ZAKAS, Nicholas C. Princípios De Orientação A Objetos Em Javascript. São Paulo: Novatec, 2022.